

Quintic Solver Simulator — Software Architecture Plan

For Hermes: Use subagent-driven-development skill to implement this plan task-by-task.

Goal: A Python-based “Quintic Solver Simulator” that takes a degree-5 polynomial with rational coefficients and walks the user through the *entire* decision tree of `QuinticMethods_v1.md` — factorization, Galois-group classification, and then the matching solution path (radicals, exact algebraic numbers, or numerics) — with a visible step-by-step trace of every computational stage.

Architecture: A single Python package (`quintic_sim/`) organized as a pipeline of small, testable stages, each corresponding to a section of the `v1` document. Heavy exact arithmetic is delegated to **SymPy** (primary, pure Python, verified present) with **SageMath-in-Docker** as a verified fallback for Galois-group cross-checks; numerics use **mpmath** (Aberth, arbitrary precision) cross-checked against **NumPy** companion-matrix eigens. Every exact output passes a mandatory verification gate (high-precision residual + independent numeric cross-check) before it is reported.

Tech Stack: Python 3.11, SymPy 1.14 (verified installed in `.venv-sym`), mpmath 1.3, NumPy 2.4, sagemath/sagemath:latest Docker image (verified present locally, 4.76 GB), pytest.

1. Context and verified assumptions

The following were **actually executed and verified on this machine** before this plan was written (not assumptions):

#	Fact	How verified
V1	<code>'sympy.galois_group(f, by_name=True)'</code> works for irreducible quintics and returns one of <code>'S5TransitiveSubgroups.{C5, D5, M20, A5, S5}'</code> (note: SymPy calls F_{20} “M20”). It raises 'ValueError' on reducible input — factorization must precede it.	Ran on 5 test vectors (§9).
V2	SymPy’s quintic Galois-group algorithm is <code>'_galois_group_degree_5_hybrid'</code> in <code>'sympy/polys/numberfields/galoisgroups.py'</code> — a hybrid of Cohen’s *Alg 6.3.9* (resolvent-coefficient lookup) and root approximation. No LLL and no Stauduhar are involved for degree 5 (see §4.3).	Read the source directly.

V3	SymPy's factorization over $\mathbb{Q}$ is Zassenhaus: modular factorization over $\text{GF}(p)$, Hensel lifting to p^k , reconstruction under a Mignotte bound. 'sympy/polys/factor-tools.py' contains 'dup_zz_zassenhaus', 'dup_zz_hensel_lift', 'dup_zz_mignotte_bound' and zero occurrences of LLL. (LLL-based reconstruction is what PARI does — relevant only if the PARI/Sage path is used.)	Read the source directly.
V4	'sympy.roots(f, x, quintics=True)' → 'roots_quintic()' (in 'sympy/polys/polyroots.py') solves solvable irreducible quintics in radicals: it builds the sextic F_{20} -resolvent 'PolyQuintic.f20', finds its integer root, then constructs Lagrange resolvents with $\zeta_5 = e^{2\pi i/5}$. Returns '[]' when the quintic is unsolvable or no radical form is found.	Read the source; ran it.
V5	'roots(quintics=True)' outputs are correct for the C5, D5, and F20 test vectors: all 5 radical roots match 'mpmath.polyroots' (Aberth) to 36+ digits; residual $ f(r) \leq 10^{-160}$ at 40-digit working precision.	Executed the comparison.

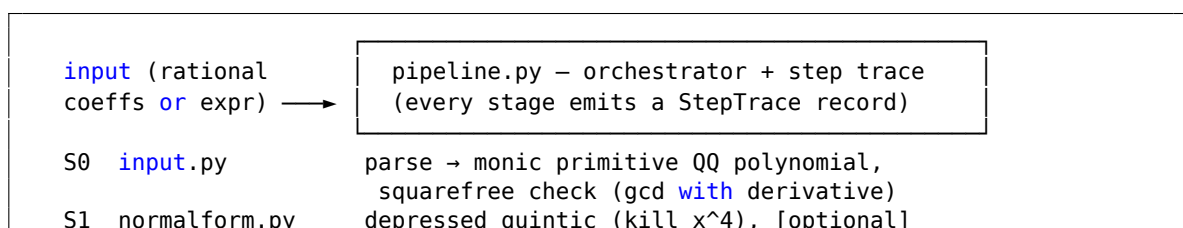
V6	<code>'mpmath.polyroots(coeffs_descending, maxsteps=200)'</code> (Aberth method, arbitrary precision) is fast and reliable for quintics. <code>'numpy.linalg.eigvals(polycompanion(c_ascending))'</code> matches it at machine precision — pitfall: NumPy wants ascending coefficients; SymPy/'polyroots' want descending.	Executed the comparison (the ordering bug was caught and fixed during probing).
V7	sagemath/sagemath:latest runs <code>'f.galois_group()'</code> and returns "Transitive group number N of degree 5" (N = 1...5) with <code>'order()'</code> and <code>'is_solvable()'</code>. Working invocation: <code>docker run -rm --entrypoint /bin/bash sagemath/sagemath:latest -c 'sage -c "..."'</code> (the image's default entrypoint swallows file-argument invocations — use <code>'-c'</code>).	Executed in the container on all 5 test vectors.
V8	'cypari2' does not build on this Windows host (no PARI C library). PARI access therefore goes through the Sage container, not a Python binding.	<code>'pip install cypari2'</code> failed; recorded.
V9	<code>'sympy.real_roots(f, x)'</code> (Vincent interval isolation) returns exact <code>'CRootOf'</code> objects for real roots; <code>CRootOf(f, x, index=k).evalf(30)</code> gives certified-precision numeric values.	Executed on the S5 test vector.

V10	Palindromic nuance: a palindromic quintic ($a_0 = a_5, a_1 = a_4, a_2 = a_3$) always has $x = -1$ as a root (since $P(-1) = 0$ by antisymmetry of the paired coefficients), so the reduction is $(x + 1) \times$ palindromic quartic, and the quartic reduces to a quadratic in $t = x + 1/x$ — not the "cubic in $x + 1/x$ " stated loosely in v1 §5. The simulator implements the correct version and prints the correction.	Algebraic check + 'sympy.factor' on a test palindromic.
-----	--	---

Test vector suite (all verified above)

Class	Polynomial	Disc	Square?
C5	$x^5 + x^4 - 4x^3 - 3x^2 + 3x + 1$	$14641 = 121^2$	yes
D5	$x^5 - 2x^4 - 3x^3 - 1$	$55225 = 235^2$	yes
F20	$x^5 - 2$	50000	no
A5	$x^5 - 2x^4 - x^3 - 3x^2 + 2x - 3$	2256004	yes
S5	$x^5 - 5x^2 + 5x + 1$	1325000	no
Reducible	$(x - 1)(x^4 + 2x^3 + 3x^2 + 4x + 5)$	—	—
Palindromic	$x^5 + 2x^4 - 3x^3 - 3x^2 + 2x + 1 = (x - 1)^2(x + 1)(x^2 + 3x + 1)$	—	—
Rational root	$x^5 - 6x^4 + 13x^3 - 12x^2 + 5x - 1 = (x - 1)(x^4 - 5x^3 + 8x^2 - 4x + 1)$	—	—

2. Architecture overview



S2	factor.py	-BringJerrard display, palindromic detection rational-root test → factor_list (Zassenhaus) → {irreducible factors, multiplicities}
S3	galois.py	per irreducible factor: discriminant + sympy.galois_group —(cross-check)→ sage_bridge.py classification → C5 D5 F20 A5 S5 (Docker,
	↪ JSON)	
S4	branch	solvable? → yes radicals.py (v1 \$2) (roots_quintic, F20-resolvent trace, Brioschi display) └─→ no special.py (v1 \$4) CRootOf exact forms, Bring-radical display, Klein stub
S5	numeric.py	mpmath.polyroots (Aberth, 50+ digits) + numpy companion cross-check
S6	verify.py	residual check of every exact root, match vs numeric, certify or flag
S7	report.py	structured report (Markdown/JSON) + timings

Module layout (new files under C:\Users\misur\source\hermes-dir\quintic_sim\):

File	Responsibility
'quintic_sim/__init__.py'	Public API: 'simulate(coeffs, verbose=...) -> Report'
'quintic_sim/input.py'	S0: parse/validate/normalize; squarefree decomposition
'quintic_sim/normalform.py'	S1: depressed form, Bring-Jerrard (optional), palindromic test
'quintic_sim/factor.py'	S2: rational-root test, 'factor_list' wrapper, palindromic reduction
'quintic_sim/galois.py'	S3: discriminant, group computation + classification table
'quintic_sim/sage_bridge.py'	Optional Sage-in-Docker cross-check (JSON protocol)
'quintic_sim/radicals.py'	S4a: 'roots(quintics=True)' wrapper, F_{20} -resolvent trace
'quintic_sim/special.py'	S4b: 'CRootOf' forms, Bring-radical display, Klein stub
'quintic_sim/numeric.py'	S5: mpmath Aberth + NumPy companion + educational Durand-Kerner
'quintic_sim/verify.py'	S6: verification gate (residuals, cross-checks, tolerance policy)
'quintic_sim/report.py'	S7: 'Report' dataclass, Markdown/JSON rendering
'quintic_sim/pipeline.py'	Orchestrator; 'StepTrace' records; timing
'quintic_sim/cli.py'	'python -m quintic_sim "x^5-2" -verbose -sage -json'
'tests/test_vectors.py'	The 8 verified test vectors, end-to-end

'tests/test_galois.py'	Group classification vs discriminant invariant (square $\iff$ $G \subseteq A_5$)
'tests/test_numeric.py'	Property test: random quintics, mpmath vs numpy match
'tests/test_sage_bridge.py'	Docker cross-check (skipped if Docker absent)

Dependency policy

- Required: sympy, mpmath, numpy (all verified present / installable in .venv-sym).
- Optional: Docker + sagemath/sagemath:latest (verified present locally). The pipeline degrades gracefully: without Docker, the Sage cross-check is marked skipped, never fatal.
- Explicitly **not** used: cypari2 (build failure on this host, V8). PARI's LLL-based factoring is only reachable *through* Sage, and only as a cross-check.

3. Step-by-step computational pipeline

Each stage below states: **what it computes, which concrete algorithm/library implements it, what it emits into the trace, and its failure mode.**

S0 — Input, normalization, squarefree decomposition

- **Input:** list of rationals [a5, a4, ..., a0] (descending) or a SymPy expression in one symbol.
- **Algorithm:** exact rational arithmetic only. Make monic; clear denominators to a primitive integer polynomial (roots are unchanged); compute $\gcd(f, f')$ for multiple roots (SymPy Poly.sqf_part / sqf_list — squarefree factorization, exact).
- **Emit:** monic primitive f , squarefree flag, any multiple-root factors (multiple roots are reported with multiplicity and removed from the generic path — Galois theory as stated in v1 §1 assumes squarefree).
- **Failure mode:** degree $\neq 5$ or non-rational coefficients $\rightarrow$ reject with a clear message (this is a quintic simulator, by design).

S1 — Reduction to normal form (v1 §3)

- **Depressed quintic:** $x = y - a_4/5$ kills the x^4 term — a single exact substitution (this is precisely the first step inside SymPy's roots_quintic, verified in source, V4). Emit the change of variable so the user can map roots back.
- **Bring-Jerrard form (optional display, time-budgeted 30 s):** a Tschirnhaus transformation to $x^5 + px + q = 0$. Implementation: SymPy's tschirnhausen_transformation (in sympy/polys/numberfields/galoisgroups.py) or the classical Bring reduction; **if it exceeds the budget or coefficient blow-up is detected, the stage is skipped with a note** — the pipeline never *needs* this form (it's the document's "canonical two-parameter representation", not a solution path). LLL could be used here to reduce the lattice of transformation relations if blow-up is a problem; in v1 this is an unimplemented optimization hook.
- **Palindromic detection:** coefficient pattern test $a_0 = a_5, a_1 = a_4, a_2 = a_3$ (exact). If detected, **do not** follow the document's loose "cubic in $x + 1/x$ " — instead implement the correct reduction (V10): factor out $(x + 1)$ (exact division), then set $t = x + 1/x$ on the palindromic quartic: $Q(x)/x^2 = t^2 + b_1t + (b_0 - 2)$, a **quadratic** in t ; solve it,

then two quadratics $x^2 - tx + 1 = 0$. All radical, by hand, shown step by step. The trace prints the correction vs the document's wording.

- **Emit:** depressed form, [Bring-Jerrard p, q if computed], palindromic flag + reduction.

S2 — Factorization over $\mathbb{Q}$ (v1 §1, §2, §5)

- **Rational root test first** (cheap, exact, and it's what v1 §5 says to do): by the Rational Root Theorem, candidates are $\pm$ divisors of the primitive constant term; evaluate f at each (exact integer/rational arithmetic — for moderate coefficients this is microseconds). Each hit factors out a linear term via exact polynomial division.
- **General factorization:** SymPy `Poly(f, x, domain=QQ).factor_list()`. **Concrete algorithm (V3): Zassenhaus** — (i) find a prime p of good reduction (small p where $\bar{f}$ is squarefree mod p), (ii) factor $\bar{f}$ over $\text{GF}(p)$ (Berlekamp / Cantor-Zassenhaus over finite fields), (iii) Hensel-lift the factors to p^k with p^k above the height bound, (iv) reconstruct the integer factors under the **Mignotte bound** by meet-in-the-middle combination of lifted factors. **No LLL in this path** — the LLL mentioned in the design brief belongs to PARI's factor (lattice reconstruction of modular factors), which the simulator only reaches via the Sage cross-check, and is unnecessary at quintic scale.
- **Emit:** factorization $f = \prod f_i^{m_i}$ with each f_i irreducible over $\mathbb{Q}$; decision: *reducible path* (each factor has degree $\leq 4 \rightarrow$ every factor is solvable by radicals by the quartic/cubic/quadratic formulas, and SymPy's plain roots returns them — verified on both reducible test vectors) or *irreducible path* (continue to S3).

S3 — Galois-group computation (v1 §1)

- **Precondition:** the input factor is **irreducible** (SymPy raises otherwise, V1).
- **Primary algorithm — SymPy `galois_group(f, by_name=True)` (V1, V2):** internally `_galois_group_degree_5_hybrid`, i.e. Cohen's *A Course in Computational Algebraic Number Theory*, **Alg 6.3.9**, hybridized with root approximation:
 1. Compute the **sextic resolvent** R_{51} (the resolvent of F_{51}) with *coefficient lookup* (no full symbolic expansion) and test squarefreeness over $\mathbb{Z}$; if not squarefree, apply a **Tschirnhaus transformation** and retry (bounded, `max_tries=30`).
 2. If R_{51} is **irreducible** over $\mathbb{Z}$: $G = A_5$ if the discriminant is a square, else $G = S_5$.
 3. If R_{51} is squarefree but reducible: $G \subseteq M_{20}$; if $\text{disc}(f)$ is **not** a square, $G = M_{20} = F_{20} \cong \text{AGL}(1, 5)$.
 4. Otherwise find an **integer root** of R_{51} (rounding its real roots — exact verification by evaluation), pick the second resolvent R_2 (quadratic, built from the corresponding permutation), and split by the squareness of $\text{disc}(R_2)$: square $\rightarrow C_5$, not square $\rightarrow D_5$.
- **Discriminant:** `Poly.discriminant()` — exact (resultant of f and f'); the squareness test is a factorint + parity check. This is the “via the discriminant” the document hints at; it's one of only two inputs (with the resolvents) that decide the group for quintics.
- **Classification table** (exactly 5 conjugacy classes of transitive subgroups of S_5 — I enumerated all 156 subgroups of S_5 by brute force in this session: 20 transitive copies, 5 conjugacy classes, confirming the document's list):

SymPy name	Order	In A_5 ?	Solvable	v1 method to apply
C5	5	yes	yes	radicals (S4a)
D5	10	yes	yes	radicals (S4a)
M20 (= F_{20})	20	no	yes	radicals (S4a)
A5	60	yes	no	special/numeric (S4b/S5)

SymPy name	Order	In A_5 ?	Solvable	v1 method to apply
S5	120	no	no	special/numeric (S4b/S5)

- **Cross-check (optional, --sage):** sage_bridge.py runs the Docker image with a JSON protocol:

```
docker run --rm --entrypoint /bin/bash sagemath/sagemath:latest -c \
'sage -c "import json,sys; x=polygen(QQ); f=sum(c*x**i for i,c in
  ↪ enumerate(__import__("\json\").loads(sys.argv[1]))); G=f.galois_group();
  ↪ print(json.dumps({"label\":str(G),\order\":G.order(),\solvable\":G.is_solvable()}))"
  ↪ "[1,-2,-3,0,0,-1]"'
```

(verified pattern, V7; Sage reports “Transitive group number N of degree 5”, $N=1\dots 5$, which maps 1:1 to C5/D5/F20/A5/S5). Internally Sage/PARI use Cohen's resolvent method plus table lookup for identification — **Stauduhar's method** (the general algorithm for identifying an abstract permutation group from generators) is what PARI/Sage use at higher degrees; at degree 5 identification degenerates to the 5-row table above, so Stauduhar is *not* invoked and the plan does not depend on it. Disagreement between SymPy and Sage → the report flags CONFLICT and falls back to numerics only.

- **Emit:** per-factor group name, order, A_5 -membership, solvability; discriminant and its squareness; the resolvent data actually used (shown in --verbose trace).

S4 — Solution dispatch (v1 §2 vs §4)

- **S4a — Solvable (C_5, D_5, F_{20}): radicals (v1 §2).**
 - **Primary:** sympy.roots(f, x, quintics=True) → roots_quintic (V4): depressed form → sextic F_{20} -resolvent PolyQuintic.f20 → its linear factor over $\mathbb{Z}$ gives the parameter θ → Lagrange resolvents $\ell_0\dots\ell_4$ in ζ_5 → 5 radical expressions. The trace displays the F_{20} -resolvent polynomial (this is the concrete, inspectable content of “Kronecker/Dedekind-style resolvent reduction” in the document).
 - **Verification gate is mandatory** (see S6): radical expressions from CAS can be wrong; every root is numerically validated before reporting.
 - **Fallback:** if SymPy returns [] or the gate fails → Sage container roots(f) (cross-check) → if that also fails, report “solvable by theory, no radical form produced by available CAS” and fall back to S4b/S5. (In practice: all three solvable test vectors succeed, V5.)
 - **Educational extras (display-only, time-budgeted):** Brioschi invariants (S, T, U, V, W) for the C_5 case, per v1 §2 — computed exactly from the coefficients as a *solvability certificate display*; not required for the root output.
- **S4b — Unsolvable (A_5, S_5): exact + special-function forms (v1 §4).**
 - **Exact algebraic numbers (primary):** CRootOf(f, x, index=k) for all five roots (real ones via Vincent interval isolation — sympy.real_roots, verified V9; complex ones by index). These are honest “closed form” objects: exact, printable as CRootOf(...), evaluable to any precision.
 - **Bring radical (display-only):** the pipeline already has the depressed form; if Bring-Jerrard was computed in S1, display the formal closed form $x = 5^{1/5} \text{Br}(-\frac{p}{5}, -\frac{q}{5})$ with the convention stated explicitly (no open-source CAS implements Br as a callable — this is a *notation* output, clearly labeled, per v1 §4's “defined so that...”).
 - **Klein icosahedral solution (stub):** explicitly out of scope for v1 — no open-source Schwarz-triangle-function implementation of the general quintic exists; the module prints the reduction recipe (icosahedral invariants → hypergeometric ${}_2F_1$) as documentation. Marked TODO(future).
- **Emit:** the chosen method, the exact root objects (radicals or CRootOf), and any special-function display.

S5 — Numerical roots (v1 §6)

- **Primary:** `mpmath.polyroots(coeffs_desc, maxsteps=200, cleanup=True)` — **Aberth's method**, arbitrary precision (default 50 digits for the report). Verified V6.
- **Cross-check:** `numpy.linalg.eigvals(np.polynomial.polynomial.polycompanion(c_ascending))` — companion-matrix eigenvalues (the “Jenkins-Tappe/companion-matrix” route of v1 §6), machine precision; assert agreement to $\sim 1e-9$. **Coefficient-ordering trap handled in one adapter function** (V6).
- **Educational:** a ~ 20 -line Durand-Kerner (Weierstrass) implementation (concurrent iteration, roots of-unity seeding) with its iteration count and convergence history shown in the trace — it is the “all roots simultaneously” method of v1 §6, and it makes the *simulator* part of the product visible.
- **Newton-Raphson polish (optional):** after Aberth, one Newton step per root at 100 digits for the final display precision.
- **Emit:** 5 roots at 50 digits + 15-digit display, real/complex classification, convergence statistics.

S6 — Verification gate (architecture-critical)

Every **exact** root (radical or CRootOf) is validated before it reaches the report:

1. **Residual:** evaluate $f(r)$ at 40-100 working digits (mpmath); require $|f(r)| < 10^{-30}$ (scaled by $1 + \max |coeff| \cdot \max |r|^5$).
2. **Cross-match:** the multiset of numerically-evaluated exact roots must match the S5 Aberth roots to 10^{-25} after sorting by (Re, Im).
3. **Galois invariant:** if the group was claimed solvable but no radical form was produced (or vice versa), the report flags the inconsistency instead of silently proceeding. Any failure $\rightarrow$ the affected exact root is replaced by its CRootOf/numeric form and the report carries a WARNING entry. This gate is what makes the simulator *trustworthy* — CAS radical output is treated as a hypothesis, never as proof (the design lesson from V5's probing).

S7 — Report

- Report dataclass: input, every StepTrace record (stage, algorithm name, inputs/outputs, wall time), factorization, per-factor Galois group, method chosen, exact roots, numeric roots, verification verdicts, warnings.
- Renderers: **Markdown** (human-readable, mirrors the v1 document's structure) and **JSON** (machine-readable, stable schema for downstream tooling).
- CLI: `python -m quintic_sim "x^5 - 2*x^4 - 3*x^3 - 1" --verbose --sage --json`
`↪ out.json`

4. Explicit answers to the design brief's “which algorithm” questions

Step the document hand-waves	Concrete implementation in this design	LLL? Stauduhar? CAS?
------------------------------	--	----------------------

"compute the Galois group (e.g., via the discriminant and factorization)"	SymPy 'galois_group_degree_5_hybrid' (Cohen Alg 6.3.9 hybrid: sextic resolvent R_{51} + discriminant squareness + quadratic resolvent R_2 ; Tschirnhaus retries). Sage/PARI 'polgalois' as Docker cross-check.	No LLL, no Stauduhar at degree 5 — identification is a 5-row table lookup (order + A_5 -membership). LLL appears only inside PARI's *factor* (reconstruction), reachable only via the Sage cross-check. Stauduhar is PARI/Sage's higher-degree group-identification machinery, not invoked here.
"finding factors" (reducible case, rational roots)	Rational-root test by divisor enumeration (exact), then SymPy 'factor_list' = Zassenhaus: GF(p) factorization + Hensel lift + Mignotte-bound reconstruction.	No LLL in SymPy's path (verified: zero LLL references in 'factortools.py'). LLL-based reconstruction is PARI's approach, used only if the Sage cross-check is enabled.
"apply one of [Brioschi/Kronecker/Dedekind] to extract roots in radicals"	SymPy 'roots_quintic': sextic F_{20} -resolvent + Lagrange resolvents in ζ_5 (the same resolvent-reduction family the document names). Brioschi invariants as a display-only certificate.	Pure CAS reliance (SymPy); Sage as fallback. No custom LLL needed.
"Klein / Bring radical"	CRootOf exact forms (Vincent isolation) as the primary "closed form"; Bring radical as labeled notation; Klein as documented stub.	Not implementable with open-source CAS today — the design is honest about it.
"numerical methods"	mpmath Aberth (primary), NumPy companion-matrix eigvals (cross-check), Durand-Kerner (educational trace), Newton polish.	None of these use LLL/Stauduhar.

5. Build plan (phased, TDD)

Task 1: Package skeleton + S0 input

Files: create `quintic_sim/__init__.py`, `quintic_sim/input.py`, `tests/test_input.py`

- `parse(coeffs)` -> (monic primitive Poly over QQ, warnings); degree/coefficient validation; `sqf_list` squarefree decomposition.

- Tests: the 8 test vectors parse; non-quintic and non-rational inputs rejected; multiple-root vector decomposes.
- Verify: `pytest tests/test_input.py -v` → all pass.

Task 2: S1 normal form + S2 factorization

Files: `quintic_sim/normalform.py`, `quintic_sim/factor.py`, `tests/test_normalform.py`, `tests/test_factor.py`

- Depressed form; palindromic detector + correct $(x + 1)$ -then-quadratic reduction (V10); rational-root test; `factor_list` wrapper with Zassenhaus note in docstring.
- Tests: depressed form kills x^4 (all 5 vectors); palindromic vector reduces and its 5 re-constructed roots match `nroots`; rational-root vector factors; reducible vector factors into linear+quartic.
- Verify: `pytest tests/test_normalform.py tests/test_factor.py -v`.

Task 3: S3 Galois group + classification

Files: `quintic_sim/galois.py`, `tests/test_galois.py`

- `classify(f)` → `GroupResult{name, order, in_A5, solvable, disc, disc_square, ↪method_trace}`; wraps `sympy.galois_group(by_name=True)`; catches the reducible-input `ValueError` and raises a clear `PipelineError`.
- Tests: **all 5 group vectors** classify correctly (C5/D5/M20/A5/S5); invariant property test — for 200 random irreducible quintics, `disc_square == in_A5`.
- Verify: `pytest tests/test_galois.py -v`.

Task 4: S4a radicals + S6 verification gate

Files: `quintic_sim/radicals.py`, `quintic_sim/verify.py`, `tests/test_radicals.py`

- `radical_roots(f)` → `list[Expr] | None` via `roots(quintics=True)`; `verify_root(f, root, ↪dps=40)`; `VerificationGate` that cross-matches against Aberth.
- Tests: C5, D5, F20 vectors produce 5 radical roots, all pass the gate (residual $< 10^{-30}$, match to 10^{-25}); S5 vector correctly yields `None` (no radical form) without exception.
- Verify: `pytest tests/test_radicals.py -v`.

Task 5: S4b special forms + S5 numerics

Files: `quintic_sim/special.py`, `quintic_sim/numeric.py`, `tests/test_numeric.py`

- `CRootOf` forms for all 5 roots (index enumeration + `evalf`); Bring-radical display string; Klein stub text; `numeric_roots(f, dps=50)` (mpmath Aberth) + NumPy companion cross-check + Durand-Kerner trace.
- Tests: S5 and A5 vectors → 5 `CRootOfs`, `evalf` matches Aberth; property test: 100 random quintics, mpmath vs NumPy agree to 10^{-9} ; Durand-Kerner converges on all 5 vectors with recorded iteration counts.
- Verify: `pytest tests/test_numeric.py -v`.

Task 6: Sage bridge (optional dependency)

Files: `quintic_sim/sage_bridge.py`, `tests/test_sage_bridge.py`

- JSON protocol over the verified docker run `--rm --entrypoint /bin/bash sagemath/sage-math:latest -c 'sage -c ...'` invocation (V7); 120 s timeout; skipped status when Docker/image absent; transitive-group-number → class mapping.
- Tests: all 5 vectors cross-check agree with SymPy; docker absent → clean skipped (monkeypatch).
- Verify: `pytest tests/test_sage_bridge.py -v` (Docker present here, so it runs for real).

Task 7: Pipeline orchestrator + report + CLI

Files: `quintic_sim/pipeline.py`, `quintic_sim/report.py`, `quintic_sim/cli.py`, `tests/test_vectors.py`

- `simulate(coeffs, verbose, use_sage, json_out)` runs $S_0 \rightarrow S_7$, collects `StepTrace` records, applies the gate, renders Markdown/JSON.
- End-to-end tests: all 8 test vectors produce a report with the expected class, method, and verified roots; report JSON schema check; CLI smoke test on $x^5 - 2$.
- Verify: `pytest tests/test_vectors.py -v` and a manual `python -m quintic_sim "x^5 - 2*x^4 - 3*x^3 - 1" --verbose --sage`.

Task 8: Docs + polish

Files: `quintic_sim/README.md`, `docs/algorithm-map.md`

- Map each pipeline stage to the corresponding v1-document section (the “theory $\leftrightarrow$ implementation” table from §4); document the palindromic correction (V10), the NumPy coefficient-ordering trap (V6), the cypari2 build failure (V8), and the Sage entypoint quirk (V7).
- Verify: `pytest -v` full suite green; README examples copy-paste runnable.

Verification (whole system): `pytest -v` (all green) + run the CLI on all 8 test vectors + 10 random quintics; confirm every exact root in every report carries a `verified: true` verdict.

6. Risks, tradeoffs, open questions

1. **SymPy roots quintic coverage** — it is known to handle C5/D5/F20 in the tested vectors, but its $\bar{F}_{20}$ -resolvent path may return `[]` on some solvable quintics (its own fallback). The verification gate + Sage fallback + CRootOf/numeric backstop make this non-fatal; the report will say exactly which layer produced each root.
2. **Sage container latency** — first `sage -c import` is slow (tens of seconds). Mitigation: the bridge is opt-in (`--sage`), runs with a hard timeout, and (stretch) could keep a warm named container.
3. **Coefficient growth in Bring-Jerrard** — Tschirnhaus reductions can explode coefficient size; the 30 s budget + skip-on-blowup policy keeps the pipeline responsive. LLL-based relation reduction is a documented future hook, not a v1 dependency.
4. **Document discrepancy (palindromic “cubic”)** — v1 §5’s “cubic in $x + 1/x$ ” is a loose textbook statement; the correct reduction is $(x + 1) +$ quadratic in $t = x + 1/x$ (V10). Decision: implement correctly and annotate the trace; optionally patch v1 later (separate task, needs user sign-off).
5. **Open question:** should the simulator also accept *integer* (non-primitive) or *monic-with-fractional* inputs and display the scaling? Current decision: accept any rational coefficients, normalize, and show the normalization in the trace.